Synthetic biology, simplified

MCSB Bootcamp — Mathematical and Computational Track

Jun Allard

A simplified version of the model from Guido, Collins et al., 2006.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
```

1. Binding and unbinding of a TF to a promoter

```
# parameters
kon = 0.01  # attachment rate, s^-1 uM^-1
koff = 0.005  # unbinding rate s^-1
C = 2.0  # microMolar, concentration of transcription factor

initial_condition = [0, 100]


def dxdt(t, state):
    """db/dt and du/dt for the promoter, given the state [bound, unbound]."""
    b, u = state

    db_dt = +kon * C * u - koff * b
    du_dt = -kon * C * u + koff * b

    return [db_dt, du_dt]


sol = solve_ivp(dxdt, [0.0, 1000], initial_condition)

T = sol.t
```

```
bound, unbound = sol.y

fig, ax = plt.subplots()
ax.plot(T, bound, "-g")
ax.plot(T, unbound, "-r")
ax.set_ylabel("Number of promoters in each state")
ax.set_xlabel("Time (seconds)")
plt.show()
```

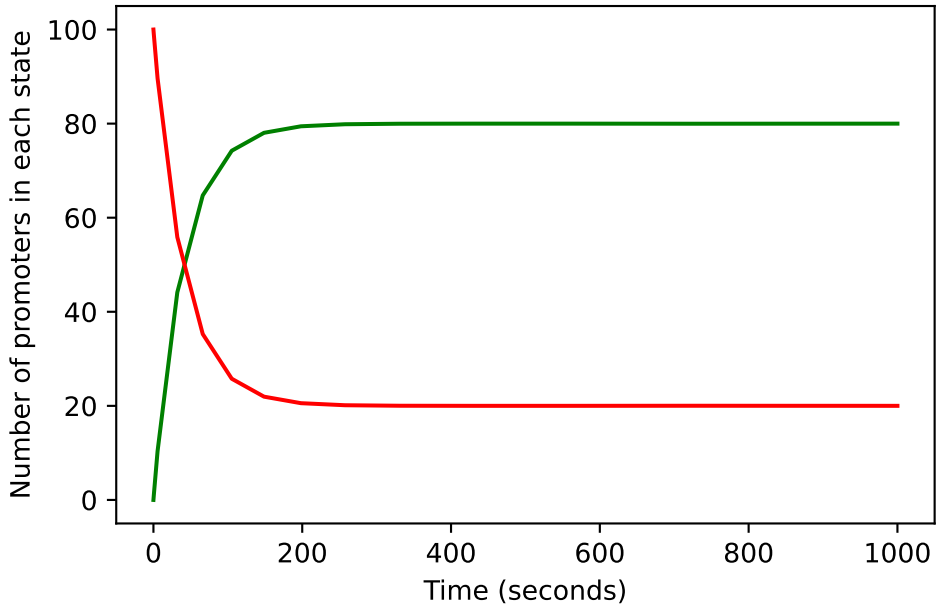

Figure 1: The same two-state model, integrated with an adaptive Runge–Kutta method. Note C has changed.

1.1 Parameter sweep

Run the model once for each of a thousand concentrations of transcription factor, and keep only the last value: the steady-state number of bound promoters.

```
param_array = np.arange(0, 10.001, 0.01)
b_storage = np.zeros(param_array.size)

for i_param, C in enumerate(param_array):
```

```
    sol = solve_ivp(dxdt, [0.0, 1000], [0, 100])

    b_storage[i_param] = sol.y[0, -1]

fig, ax = plt.subplots()
ax.plot(param_array, b_storage, "-b")
ax.set_xlabel("Concentration of transcription factor (uM)")
ax.set_ylabel("Number of bound promoters")
# ax.set_xscale("log")
plt.show()
```

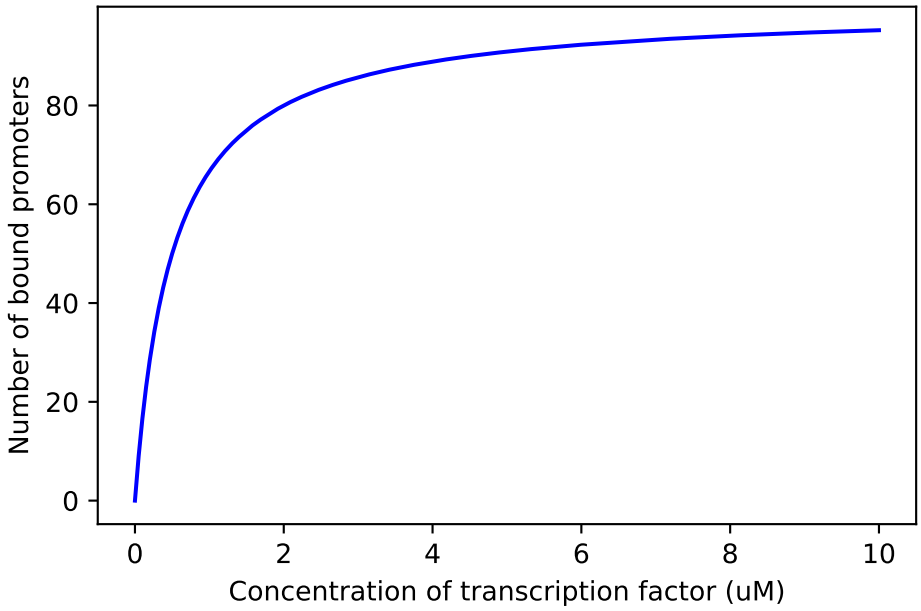

Figure 2: Steady-state bound promoters against transcription factor concentration.

2. mRNA and protein

2.1 Creation and destruction of mRNA and protein

mRNA is made at a constant rate and degrades. Protein is made in proportion to the mRNA present and degrades.

```
# parameters
gamma_m = 0.2
```

```
delta_m = 0.02
gamma_p = 0.04
delta_p = 0.02

initial_condition = [0, 0]


def dxdt(t, state):
    """dm/dt and dp/dt for the mRNA-protein system, given the state [mRNA, protein]."""
    m, p = state

    dm_dt = +gamma_m - delta_m * m
    dp_dt = +gamma_p * m - delta_p * p

    return [dm_dt, dp_dt]


sol = solve_ivp(dxdt, [0.0, 600], initial_condition)

T = sol.t
mrna, protein = sol.y

fig, ax = plt.subplots()
ax.plot(T, mrna, "-r")
ax.plot(T, protein, "-", color=[0.5, 0, 1])
ax.set_ylabel("Concentration of RNA (red) and product (purple)")
ax.set_xlabel("Time (seconds)")
plt.show()
```

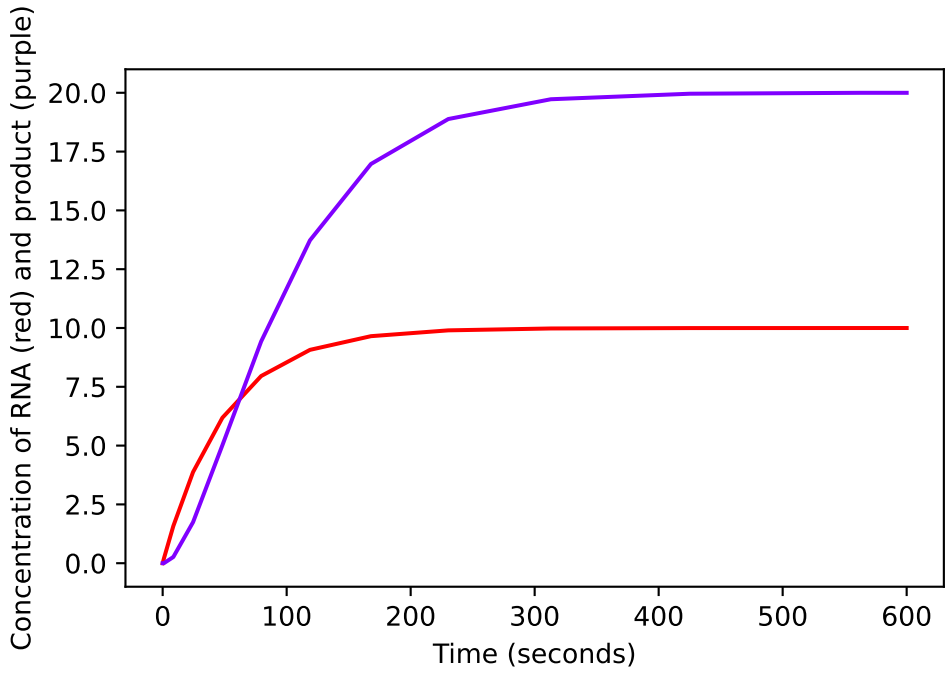

Figure 3: mRNA and protein rising to their steady states, the protein lagging behind.

2.2 Combine the RNA model with promoter binding state

The promoter is the same two states as in section 1 — empty, or activator bound — and the rate at which mRNA is produced depends on which of the two it is in.

```
# parameters
kon = 0.001  # s^-1 uM^-1
koff = 0.0005  # s^-1

delta_m = 0.05
gamma_p = 0.02
delta_p = 0.01

C = 10  # concentration of activatory transcription factor (uM)

initial_condition = [0, 1, 0, 0]


def dxdt(t, state):
    """The two promoter states, then mRNA and protein."""
```

```
    b, u, m, p = state

    db_dt = +kon * C * u - koff * b
    du_dt = -kon * C * u + koff * b

    # how fast mRNA is made, given which state the promoter is in.
    # Empty still transcribes, so the range is a factor of two: 1.0 unbound, 2.0 with the act
    gamma_m = 1.0 * u + 2.0 * b

    dm_dt = +gamma_m - delta_m * m
    dp_dt = +gamma_p * m - delta_p * p

    return [db_dt, du_dt, dm_dt, dp_dt]


sol = solve_ivp(dxdt, [0.0, 1000], initial_condition)

T = sol.t
bound, unbound, mrna, protein = sol.y

fig, (ax_promoter, ax_product) = plt.subplots(2, 1, figsize=(6, 7))

for state, label in zip((unbound, bound), ("Empty", "activator bound")):
    ax_promoter.plot(T, state, label=label)
ax_promoter.set_ylabel("Number of promoters in each state")
ax_promoter.set_xlabel("Time (seconds)")
ax_promoter.legend()

ax_product.plot(T, mrna, "-r")
ax_product.plot(T, protein, "-", color=[0.5, 0, 1])
ax_product.set_ylabel("Concentration of RNA (red) and product (purple)")
ax_product.set_xlabel("Time (seconds)")

fig.tight_layout()
plt.show()
```

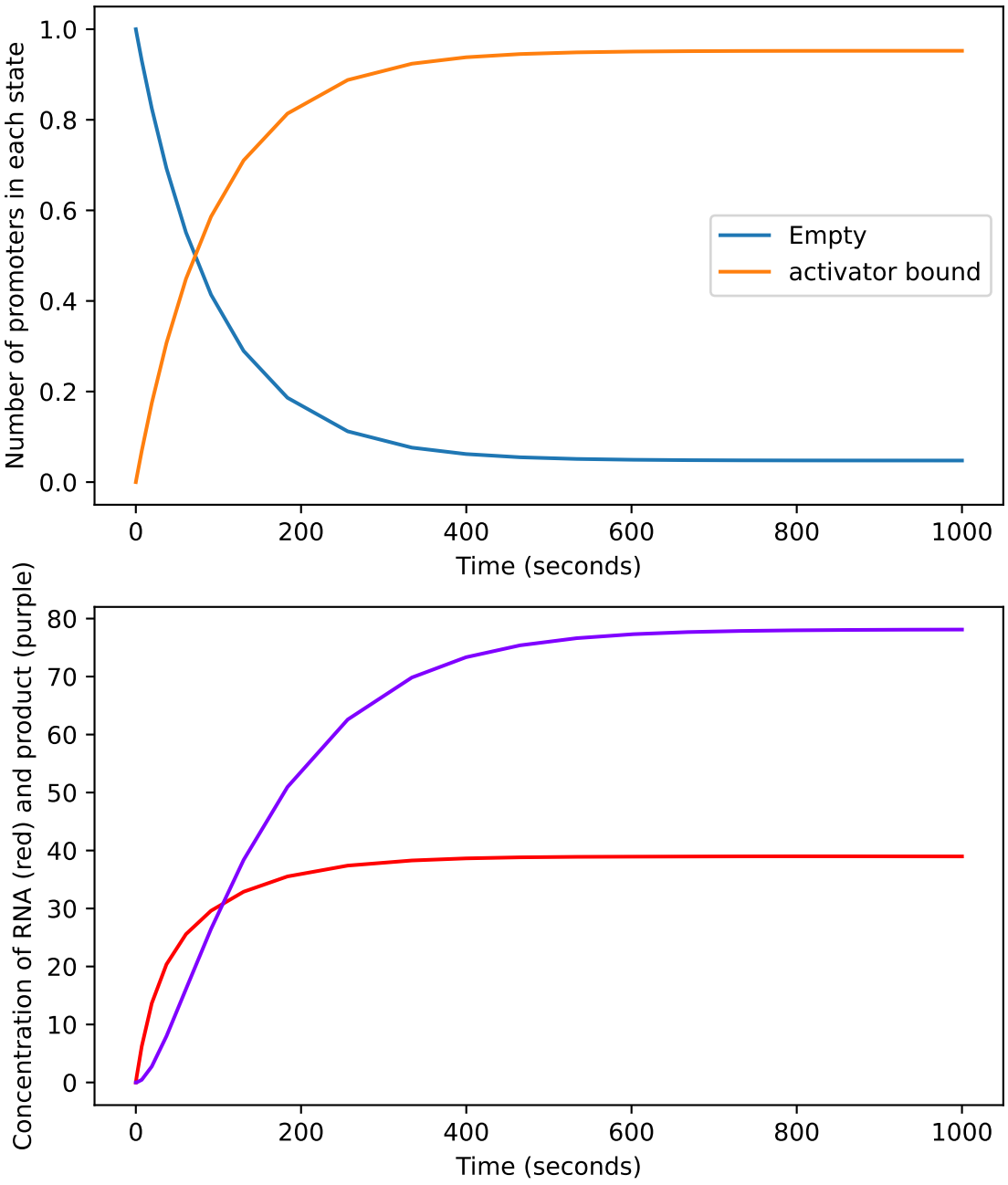

Figure 4: Above, the promoter moving between its two states; below, the mRNA and protein that follow from it.

2.3 Parameter sweep across activator concentration

Same sweep as 1.1, but over the concentration of activating transcription factor, and now reading off the steady-state protein.

```
param_array = np.linspace(0, 5, 200)


g_storage_no_feedback = np.zeros(param_array.size)

for i_param, C in enumerate(param_array):  # concentration of activatory TF (uM)

    sol = solve_ivp(dxdt, [0.0, 10000], [0, 1, 0, 0])

    g_storage_no_feedback[i_param] = sol.y[3, -1]

fig, ax = plt.subplots()
ax.plot(param_array, g_storage_no_feedback, "-b")
ax.set_xlabel("Concentration of activating TF (uM)")
ax.set_ylabel("Concentration of product (uM)")
ax.set_ylim(0, 100)
# ax.set_xscale("log")  # uncomment together with the logspace sweep above
plt.show()
```

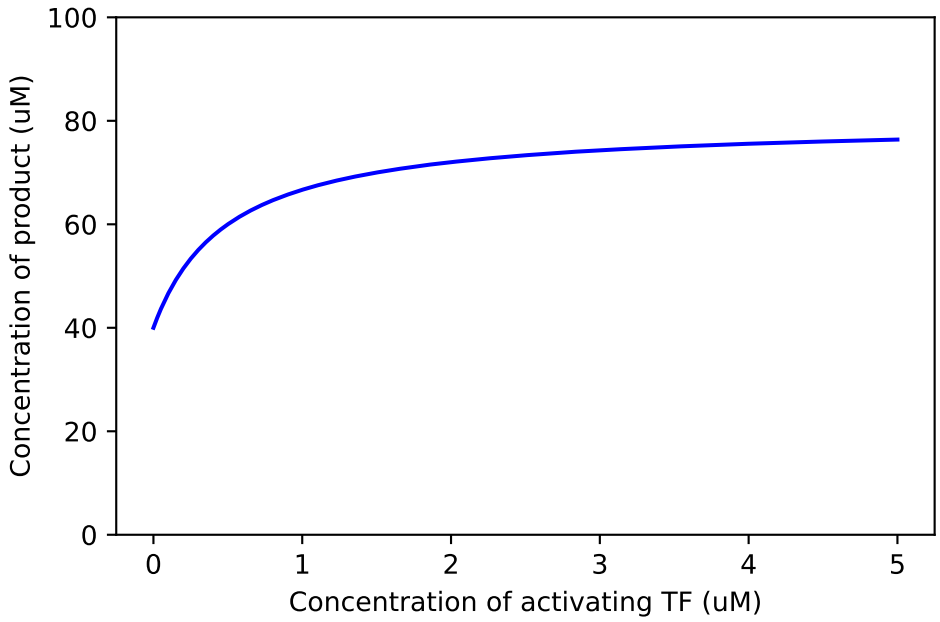

Figure 5: Steady-state product against activator concentration. The half-way point sits at the dissociation constant, 0.5 uM.

2.4 Positive feedback

Add a second gene on the same promoter whose protein is itself an activator of that promoter. The state grows from four variables to six.

The curve from 2.3 is drawn underneath in pale grey, so the effect of the feedback can be read off one pair of axes rather than by flipping between two figures. Both sweeps have to run over the same `param_array` for that to line up, so changing the range here means changing it in 2.3 as well.

```
# Same sweep as 2.3, and it has to stay the same for the two curves to line up.
param_array = np.linspace(0, 5, 200)


g_storage = np.zeros(param_array.size)

# parameters
kon = 0.001  # s^-1 uM^-1
koff = 0.0005  # s^-1
```

```
delta_m = 0.05
gamma_p = 0.02
delta_p = 0.01

gamma_p2 = 1e-4


def dxdt(t, state):
    """The two promoter states, then two mRNA-protein pairs, the second feeding back on the p
    b, u, m, p, m2, p2 = state

    activator = C + p2

    db_dt = +kon * activator * u - koff * b
    du_dt = -kon * activator * u + koff * b

    gamma_m = 1.0 * u + 2.0 * b

    dm_dt  = +gamma_m - delta_m * m
    dp_dt  = +gamma_p  * m - delta_p * p

    dm2_dt = +gamma_m - delta_m * m2
    dp2_dt = +gamma_p2 * m2 - delta_p * p2

    return [db_dt, du_dt, dm_dt, dp_dt, dm2_dt, dp2_dt]


for i_param, C in enumerate(param_array):  # external activatory transcription factor

    sol = solve_ivp(dxdt, [0.0, 10000], [0, 1, 0, 0, 0, 0])

    g_storage[i_param] = sol.y[3, -1]

fig, ax = plt.subplots()
# 2.3's curve first, so the feedback one is drawn on top of it rather than behind it.
ax.plot(param_array, g_storage_no_feedback, "--", color="0.7", label="No feedback (2.3)")
ax.plot(param_array, g_storage, "-b", label="Positive feedback")
ax.set_xlabel("Concentration of activating TF (uM)")
ax.set_ylabel("Concentration of product (uM)")
ax.set_ylim(0, 100)
ax.legend()
# ax.set_xscale("log")  # uncomment together with the logspace sweep above
```

```
plt.show()
```

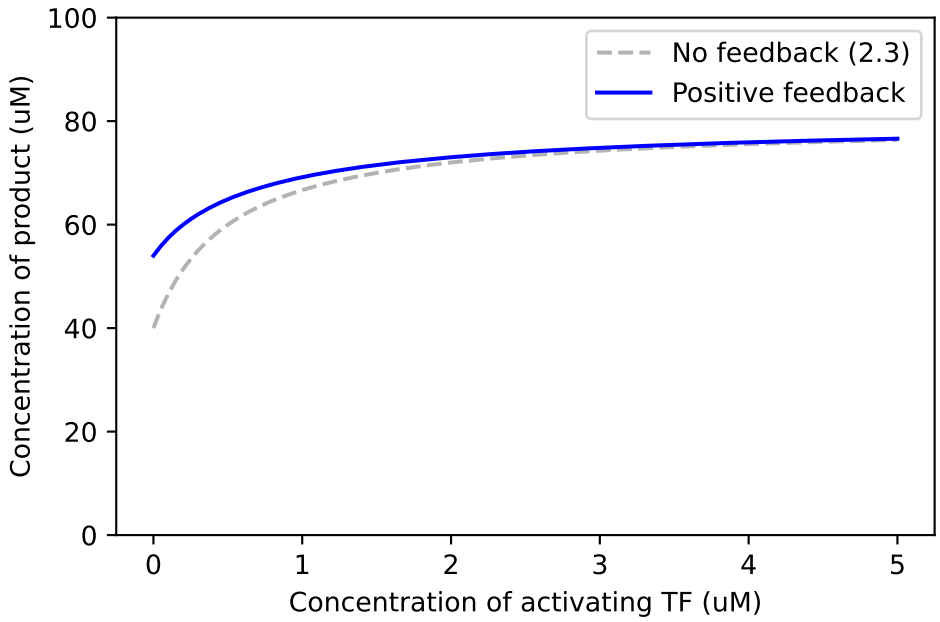

Figure 6: The same sweep with positive feedback added, over the curve from 2.3. The feedback lifts the floor and moves the half-way point left, from 0.5 uM to 0.2 uM.